defiro · QA Strategy

Nov 5, 2026 · @Duarte

Scope and quality goals

A user story is done when its agreed acceptance tests are automated and green in a pipeline that blocks any change that breaks them. This strategy applies that rule to the 15 MVP user stories, following the practices of the TQS lectures TP01 to TP07.

It covers the two Spring Boot services (Workflow Designer and Workflow Engine), the web front end, the PostgreSQL databases and the plugins. The identity provider, the email service and the municipal systems are tested through test doubles and contract tests. The CMD signature is out of scope because it is not in the MVP.

Quality goal	How it is checked	Target
Functional correctness	Cucumber scenarios for every acceptance criterion	Every criterion has at least one green scenario
Reliability and security	SonarQube Quality Gate on every pull request	No new issues; overall ratings A
Maintainability	Quality Gate, code review, refactoring under tests	New-code coverage of at least 80 %, duplication of at most 3 %
Accessibility	Automated audit (axe-core) and keyboard-only browser journeys	WCAG 2.2 AA, no new violations
Performance	k6 load test with thresholds	Proposed SLO: 95 % of requests under 300 ms, to confirm
Usability	Usability sessions with the SUS questionnaire	Findings logged and triaged each iteration

From story to done

The team agrees on each story's acceptance tests before writing its code (ATDD). The tests then drive the work and say when the story is done. Each story is a card (who, what, why), a conversation (the detail) and a confirmation (its acceptance tests).

Every story goes through the same four steps in each sprint:

1. **Discuss**, in backlog refinement before the sprint. The three amigos (business, development, testing) run Example Mapping on the story: rules, examples and open questions. The session takes about 25 minutes.
2. **Distill**, before or at sprint start. Examples become Gherkin scenarios in the story's feature file.
3. **Develop**, during the sprint. The scenarios start red, unit tests are written test-first underneath, and the scenarios turn green (the double loop).
4. **Demo**, at the sprint review. The working story is shown with its green scenarios, and the feedback goes back to step 1.

Reading the Example Mapping table:

- **Many open questions:** the story is not ready. Find the answers first.
- **Many rules:** the story is too big. Split it.
- **Many examples under one rule:** the rule is complex. Look at it again.
- **Few cards and quick agreement:** the story is ready to build.

Acceptance criteria are specific to each story. The Definition of Done is the same quality bar for every story (see below).

AI agents. An agent may draft examples, scenarios and step definitions from a story. The team audits the draft for invented rules, weak Thens and UI steps. Feature files change only through human review, and an agent is never allowed to edit a scenario to turn a build green.

Acceptance tests from the user stories

Each user story becomes one Cucumber feature file. Each acceptance criterion becomes a `Rule:`, and each example agreed in Example Mapping becomes a scenario. The chain runs from story to rules, examples, scenarios, executions on every build, and finally the story's status. It answers three questions: which stories are done, which rule last night's build broke, and which rules have no test.

The 15 user stories and their acceptance criteria are in the last section of this document.

How scenarios are written

- One feature file per story, tagged with its use case, for example `@UC18`. The story's «As a / I want / so that» is the feature's description.
- Declarative, in business words: what the user does, never clicks or field names.
- One `When` per scenario and an exact `Then`: which state, which message, which person.

- Independent: each scenario sets its own context, with a `Background:` for shared givens.
- Boundary values become a `Scenario Outline` with an `Examples:` table, for example a deadline of 0, 1, 90 and 91 working days.
- An open question stays as a `# open question:` comment, untested until it is answered.
- Other tags: `@smoke` for the post-deployment checks and `@wip` for work in progress, which the pipeline excludes.
- Scenarios are in English, like the code; the values the user sees stay in Portuguese.

Worked example: UC18, after Example Mapping

Rule (acceptance criterion)	Examples
At least one reason must be chosen	No reason → nothing is sent · two reasons → both are recorded
A return needs a message when no field has a note	No note, no message → the message is asked for · one note → returned without a message
Rejecting ends the procedure	Final reason → the procedure ends as rejected and the requester sees the reason

Open questions: can a rejected request be reopened? How long does «Desfazer» stay available?

```
@UC18
Feature: Return or reject a request
  As a Reviewer,
  I want to return a request for correction or reject it, choosing one or
more reasons,
  so that the requester knows exactly what went wrong.

  Background:
    Given request "AC-2026-0147" from Ana Ferreira is waiting for Rui Matos's
review

  Rule: A return needs a message when no field has a note

    Scenario: Return with no note and no message
      When Rui Matos returns the request for "Incorrectly filled" with no
message
      Then the request is still waiting for Rui Matos's review
      And Rui Matos is asked for a message
```

```
  Rule: Rejecting ends the procedure

    Scenario: Reject for a final reason
      When Rui Matos rejects the request for "Trip not authorised"
      Then the procedure ends as rejected
      And Ana Ferreira sees the reason "Trip not authorised"

  # open question: can a rejected request be reopened?
```

Where each story's scenarios run. Each scenario runs at the cheapest level that shows the behaviour, usually the REST API with a real database. Only five user journeys go through the browser, because a browser test should catch a defect that no lower test can.

Story	Scenarios run at	Browser journey
UC01 Authenticate	Engine API, with a fake OIDC identity provider	Sign in and land on the inbox
UC03 Create workflow	Designer API	Create a workflow from name to «Workflow criado»
UC04 Form fields	Designer API; validation rules in `@WebMvcTest`	Part of the UC03 journey
UC05 Group per step	Designer API, with the directory stubbed	Part of the UC03 journey
UC06 Deadlines	Domain (working-day calculation) and Designer API	Part of the UC03 journey
UC13 Start a procedure	Engine API	Ana submits a travel allowance request
UC15 Attach files	Engine API, with object storage in Testcontainers	Part of the UC13 journey
UC17 Approve	Engine API	Rui returns with a note, Ana corrects, Rui approves
UC18 Reject	Engine API	None
UC19 Notes	Engine API	Part of the UC17 journey
UC20 Correct	Engine API	Part of the UC17 journey
UC22 Validate and close	Engine API	Joana closes the request and downloads the document

Story	Scenarios run at	Browser journey
UC23 Download	Engine API, checking the PDF's content	Part of the UC22 journey
UC25 Audit log	Engine API; history queries in `@DataJpaTest`	None
UC27 Inbox	Engine API	Part of the UC17 journey

Test levels and tools

The suite follows the test pyramid: many fast unit tests, fewer integration and API tests, and a handful of browser journeys. Each test lives at the narrowest level that can answer its question. The quadrant column places each level in the agile testing quadrants (Q1 to Q4).

Level	What it checks in defiro	Tools	Quadrant	Pipeline stage
Unit	Domain rules: deadlines, validation, reason types, step order	JUnit 6, AssertJ, Mockito	Q1	Local and commit
Spring slices	HTTP contract and validation (`@WebMvcTest`); queries (`@DataJpaTest`)	Spring Boot Test	Q1	Commit
Integration	Repositories, the plugin manager, object storage, on the real engines	Testcontainers (PostgreSQL)	Q1	Acceptance
API	Each service's REST endpoints over real HTTP	REST Assured	Q2	Acceptance
Acceptance (BDD)	Every acceptance criterion of the 15 stories	Cucumber on the JUnit Platform	Q2	Acceptance

Level	What it checks in defiro	Tools	Quadrant	Pipeline stage
Browser journeys	The five end-to-end journeys, including keyboard-only use	Playwright for Java	Q2	Extended
Accessibility	WCAG 2.2 AA on every screen and state	axe-core through Playwright	Q4	Extended
Load	Response times against the SLOs	k6, with thresholds	Q4	Extended
Smoke	The deployment is up, reaches the database and answers the main flow	`@smoke` scenarios, health endpoint	Q2	Staging and production
Exploratory and usability	What no script anticipated; ease of use with real people	Test charters; sessions with the SUS questionnaire	Q3	Each iteration, by hand

Rules for every automated test

- **Tests come from the contract.** Write one case per equivalence class and test both sides of every boundary.
- **Readable and deterministic.** Use arrange, act, assert, with one behaviour per test. A fixed clock replaces real time, so working-day deadlines can be tested.
- **Doubles only for what the team does not control:** time, the identity provider, email, the municipal systems. Stub queries and verify commands. An integration test then checks what each double assumed.
- **The real database engine,** through Testcontainers. An embedded database is itself a fake.
- **Browser tests locate elements like a user does:** by role, label and text. This also checks the accessible names. They never sleep; they wait for something observable.
- **Coverage shows untested code. It does not prove anything was checked:** every test needs an assertion that a wrong implementation would fail.
- **Tests drafted by AI are audited like any other code.** The team owns the oracle. The review question is: «what would make this test fail?»

Code quality

Every pull request must pass a SonarQube Quality Gate and an approving review before it can be merged. Static analysis finds patterns, tests find behaviour, and review finds what neither describes, so the strategy uses all three.

Static analysis in three places

- **While coding:** SonarQube for IDE, in connected mode, with the same rules as the server.
- **On every pull request:** the CI quality job runs the build, the tests and JaCoCo coverage, then the SonarQube scanner with `sonar.qualitygate.wait=true`. A failed gate fails the job.
- **On the server:** SonarQube Cloud keeps the history, the dashboard and the pull-request status. Its free tier covers public projects.

Quality Gate. All the code is recent, so it is held to the bar as a whole. This follows the course-project example: the Sonar way conditions on new code, plus conditions on the overall code.

Condition	Applies to	Fails when
Issues	New code	More than 0 new issues
Coverage	New code	Below 80 %
Duplicated lines	New code	Above 3 %
Reliability rating	Overall code	Worse than A
Security rating	Overall code	Worse than A
Coverage	Overall code	Below 70 % (proposed value, to confirm and justify in the report)

The Sonar way ignores coverage and duplication on changes under 20 lines, so a small hotfix still needs a test in review.

Using the numbers with judgement

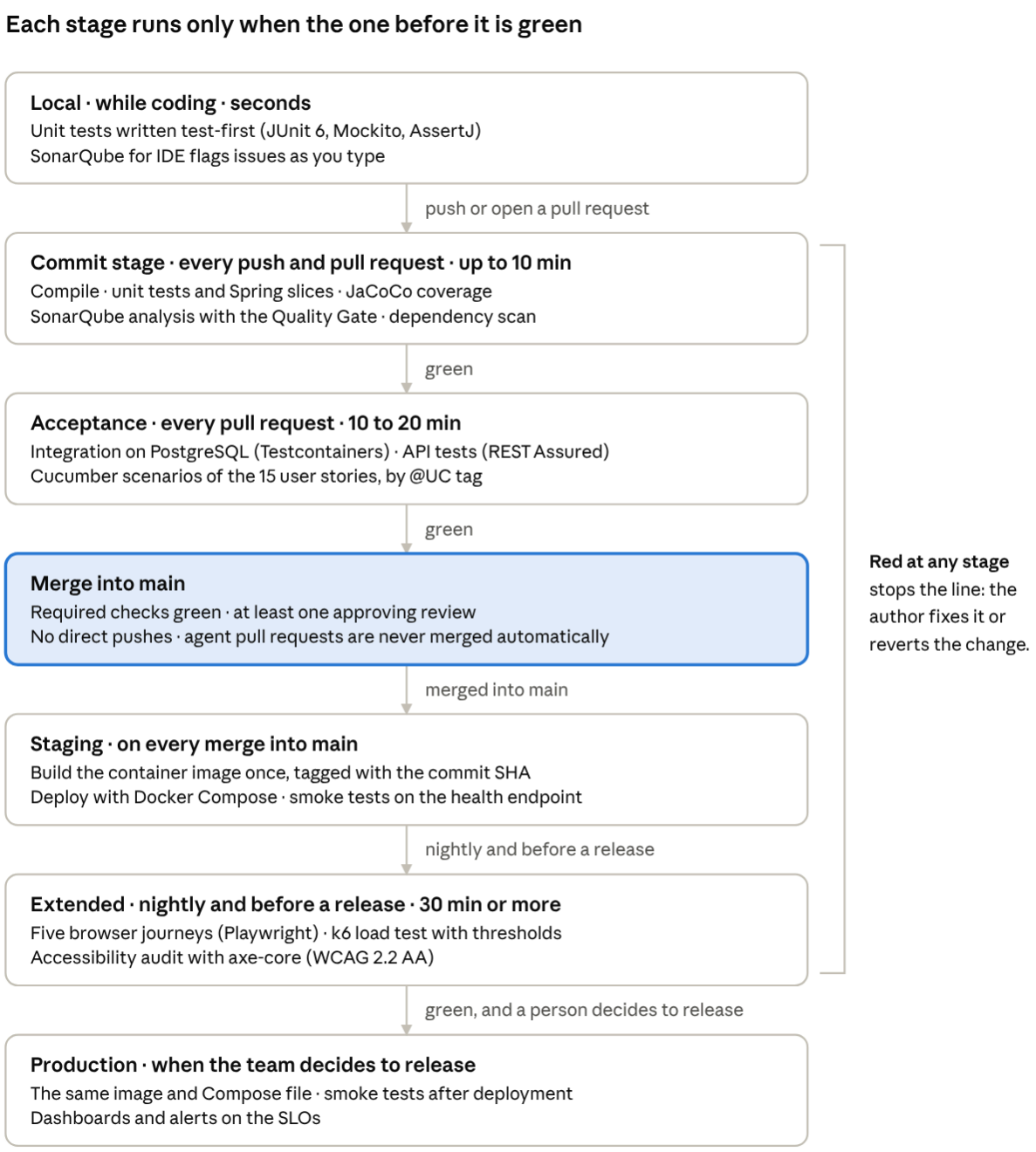
- Every accepted issue or false positive has a written reason. Issues are never silenced in bulk to turn the gate green.
- A test without an assertion written only to reach 80 % is rejected in review.
- The quality profile and the gate belong to the team and are revised each iteration.

Code review

- Pull requests are small, and branches live for hours or a couple of days, never a whole sprint. Each branch is named after its story, for example `UC18-reject-reasons`.
- Each pull request needs at least one approving review. The reviewer reads the tests first and asks what would make them fail.
- Code smells found in review or by the analyser are refactored in small steps, with the tests as the safety net.

Continuous integration and delivery pipeline

Every change rides the same GitHub Actions pipeline, and main is protected so that only a green, reviewed change reaches it. Fast, cheap tests run on every push; slower ones run later.



pipeline · 7 stages, each gated on the one before

The image is built once and promoted unchanged from staging to production. Deploying is not releasing: a person decides when a version goes live.

Pipeline rules

- Before any story, build a walking skeleton: one endpoint, one test, the CI workflow, a container image and one deployment.
- Protect main: changes only through pull requests, required checks (build, quality, acceptance), at least one approving review, the branch up to date, and no force-push.
- Least privilege: workflow tokens are read-only unless a job needs more, secrets live in GitHub and never in the code, and third-party actions are pinned to a version.

- Test reports and Cucumber results are uploaded even when the tests fail.
- Agents in CI may read a failed run and propose a fix, with read-only rights. People decide merges and releases.

Definition of Done

A user story is done only when all of these hold. The pipeline enforces most of them.

1. Every acceptance criterion is a `Rule:` with at least one scenario, tagged with the story, and none is left as `@wip`.
2. The story's scenarios, unit tests and integration tests are green in CI.
3. The Quality Gate passes, with a written reason for every accepted issue.
4. The code was reviewed and approved in a pull request, and merged into main.
5. The change is deployed to staging and the smoke tests pass.
6. Any browser journey the story touches is updated and green in the nightly run.
7. The accessibility audit finds no new WCAG 2.2 AA violation on the screens the story changed.
8. The story was demonstrated at the sprint review and accepted.

Defects and quality metrics

A red build is fixed before anything else, and every defect first becomes a failing test. A green check is evidence, not proof, so the team measures to learn and never treats a metric as a target.

When something breaks

- **Stop the line.** Nobody pushes on top of a red main. Whoever broke it fixes it straight away, and if the fix is not obvious within minutes, reverts and fixes it on a branch.
- **A flaky test is a defect.** It is quarantined, fixed or deleted, and never just re-run.
- **Every defect starts with a test.** A defect found in a test, a review, an exploratory session or production becomes an issue linked to its story. A test that reproduces it is written first, then the fix.
- **Usability and exploratory findings** (Q3) are logged as issues and triaged at the next refinement.

What the team watches

Metric	Source	Question it answers
Stories done	Cucumber results grouped by `@UC` tag	Which stories have all their scenarios green?
Rules without tests	Feature files compared with the stories' acceptance criteria	Which agreed rule is still untested?
Quality Gate status and coverage on new code	SonarQube	Did the last change leave the code better?
Commit-stage duration	GitHub Actions	Is feedback still under about 10 minutes?
Deployment frequency, lead time, change failure rate, time to restore	Pipeline and deployment history (DORA)	Is delivery getting faster without getting less stable?
Response times against the SLOs	k6 thresholds and the dashboards	Does defiro keep its performance promise under load?

User stories and acceptance criteria

These are the 15 MVP user stories, the same as in the MVP User Stories document. Each acceptance criterion becomes a `Rule:` in the story's feature file.

UC01 · Authenticate (SSO Entra ID)

As a municipal employee, **I want** to sign in with my organization's Microsoft account, **so that** I don't need another password and only see what my role allows.

Acceptance criteria

1. The home page has a single «Entrar» button, which opens the organization's sign-in page.
2. defiro stores no password. Name, unit, job title and manager are read from Entra ID.
3. After signing in, the user lands on their task inbox (UC27).
4. The side menu only shows what the user's role allows: «Workflows» and «Criar workflow» appear only for the Workflow Manager.
5. The top bar shows the user's name and job title.
6. Signing out ends the session and returns to the home page with the notice «Sessão terminada».

UC03 · Create workflow

As a Workflow Manager, **I want** to create a workflow in the designer, one question at a time, **so that** a new type of request can be made without programming.

Acceptance criteria

1. «Criar workflow» is the main button in the Workflow Manager's side menu.
2. The designer is a sequence of screens: name, who can request, path, request form, review and create.
3. Each screen shows «Passo X de N» and a progress bar.
4. Each review step is set up in three screens: name and who handles it (UC05), deadline (UC06), and reasons to return and to reject.
5. Each step needs at least one reason, and every reason needs a name. Return reasons and reject reasons are separate lists.
6. The path so far is always visible. Steps can be edited, moved up or down, removed or added (review or final validation).
7. The user cannot move on while an answer is missing or invalid. The error appears next to the field.
8. «Anterior» goes back without losing answers, and «Sair» leaves the designer.
9. On the last screen, creating the workflow shows the notice «Workflow criado».
10. The new workflow then appears in «Workflows» and in «Novo pedido» for the people allowed to request it.

UC04 · Define the form fields of each step

As a Workflow Manager, **I want** to choose the questions the request form asks, **so that** the platform generates the form without programming.

Acceptance criteria

1. Name, organizational unit and manager are filled in from the profile and are never asked.
2. Each field is created on its own screen with a label, an answer type and whether it is mandatory.
3. The answer type is chosen from cards with examples: short text, long text, number, date, date and time, yes or no, single choice and files.
4. A single-choice field needs at least two options.
5. A field with no label, or a choice with no options, is flagged in the field list.
6. Fields can be moved up or down and removed.

7. The workflow cannot be created without at least one question for the requester. The message says: «Formulário do pedido: ainda não tem perguntas para quem faz o pedido… crie pelo menos uma pergunta, por exemplo «Material pedido»».
8. In the generated form, optional fields say «(opcional)», number fields only take digits and dates are written as dd/mm/aaaa.

UC05 · Assign a group or unit to each step

As a Workflow Manager, **I want** to say which group or unit handles each step, **so that** each task reaches the right people.

Acceptance criteria

1. Each step's handler is chosen from the organization's groups and units, for example «Chefias da unidade do requerente» or «Divisão Financeira».
2. The first screen sets who can request, for example «Todos os trabalhadores». The workflow only appears in «Novo pedido» for those people.
3. A step with no handler cannot move on.
4. The path shows who handles each step.
5. When a request reaches a step, the task appears in the inbox of the people in that step's group (UC27).

UC06 · Set legal deadlines for each step

As a Workflow Manager, **I want** to set the legal deadline of each step, **so that** participants know how long they have to act.

Acceptance criteria

1. Each step has a mandatory deadline in working days, from 1 to 90.
2. The help text «De 1 a 90 dias úteis» is shown before anything is typed.
3. A value outside that range, such as 200, shows an error straight away.
4. The workflow summary shows the total deadline in working days.
5. In the inbox, each task shows its due date, calculated from the step's deadline (UC27).

UC13 · Start a procedure

As a Requester, **I want** to fill in a request a few questions at a time, **so that** I submit it complete and correct without feeling overwhelmed.

Acceptance criteria

1. «Novo pedido» only lists the workflows the user is allowed to start.

2. The form is a sequence of screens with «Passo X de N» and a progress bar. The travel allowance has 8 steps: details, trip, dates, certificate, transport, expenses, other documents and review.
3. Name, unit and manager are pre-filled from the profile.
4. NIF and IBAN are typed, checked, and grouped as they are typed.
5. Mandatory fields have an asterisk (*), and so does every field another one depends on. Optional fields say «(opcional)».
6. «Continuar» only moves on when the step is correct. Otherwise the error appears next to the field and the focus goes to it. Enter also continues.
7. Dates are written as dd/mm/aaaa and times as hh:mm (24 hours), with a calendar button next to each date. A return before the departure is flagged straight away.
8. If the certificate dates do not match the trip, a red warning appears. The user can still continue, and the request is flagged at final validation (UC22).
9. «Anterior» goes back without losing answers, and «Sair» leaves the form.
10. The last screen sums up every answer, with «Alterar» for each part.
11. Submitting shows the notice «Pedido submetido» with the reference, for example AC-2026-0147. The request goes to the first step's group.

UC15 · Attach supporting files

As a Requester, **I want** to attach the receipt of each expense and my certificate of attendance, **so that** the reviewers can check every amount and date.

Acceptance criteria

1. An expense can only be added together with its receipt. Type, date, amount and receipt are all mandatory.
2. An expense date must fall within the trip dates. The allowed range is shown under the field.
3. All errors of an expense appear together when adding it, each next to its field.
4. If the trip dates change and an expense falls outside them, that row is marked «Fora das datas da deslocação» and the request cannot be submitted.
5. The certificate of attendance has its own space. Its start date, end date and file are mandatory, and the end cannot be before the start.
6. Any other file goes in «Outros documentos», which is optional.
7. Attached files can be removed, and each change shows a short notice, for example «Ficheiro junto».
8. Each expense row shows its receipt, and the reviewer can open it from that row.

UC17 · Approve a document

As a Reviewer, **I want** to approve a request in a couple of clicks from my inbox, **so that** it moves on without delay.

Acceptance criteria

1. A request opens from the inbox in one click. It shows the data, the files, the path with the current step and the history.
2. On a resubmitted request, the reviewer sees what changed in the new version.
3. Approve is the main button and comes last in its group.
4. Approving takes 2 clicks from the inbox, with no confirmation dialog.
5. Approving sends the request to the next step and its group, and the reviewer returns to the inbox.
6. The notice «Pedido aprovado» appears with «Desfazer», which reverses the decision. The notice stays until it is closed.
7. The approval is logged in the history with the reviewer's name, job title, date and time (UC25).

UC18 · Reject a document

As a Reviewer, **I want** to return a request for correction or reject it, choosing one or more reasons, **so that** the requester knows exactly what went wrong.

Acceptance criteria

1. Return and Reject are two separate buttons.
2. Each button opens its own list of the reasons defined in the workflow, as checkboxes. At least one reason must be chosen, and more than one can be.
3. Return only lists correctable reasons. It sends the request back to the requester for correction (UC20).
4. Reject only lists the reasons that end the request. Its confirm button is red, and confirming ends the procedure as rejected.
5. When returning, the message is mandatory if the reviewer left no note on any field (UC19).
6. Choosing «Datas do certificado não coincidem» fills in the message with the dates that do not match.
7. The notice reads «Pedido devolvido para correção» or «Pedido rejeitado», with «Desfazer».
8. The chosen reasons appear in the history, in the notice, and in what the requester sees when correcting.
9. Rejecting needs no signature.

UC19 · Add notes to a document

As a Reviewer, **I want** to leave a note on the exact field that is wrong, **so that** the requester knows what to fix and where.

Acceptance criteria

1. The reviewer can add a note to any field of the request.
2. The note field is mandatory: saving an empty note shows an error instead of discarding it.
3. Notes can be edited and deleted before deciding. Each change shows a short notice, for example «Nota guardada».
4. Notes go to the requester together with a return (UC18).
5. The requester sees each note next to its field, with the reviewer's name and job title.

UC20 · Correct a returned document

As a Requester, **I want** to fix only what the reviewer pointed out and send the request again, **so that** I don't have to start over.

Acceptance criteria

1. A returned request appears in the requester's inbox, marked as returned and showing the reasons.
2. It opens on the summary screen, with the parts to correct highlighted.
3. Each reviewer note sits next to its field, with «Ir para o campo».
4. The answers already given are kept, and the same checks as UC13 apply.
5. Resubmitting creates a new version of the request and shows the notice «Pedido reenviado».
6. The procedure starts again from the first step. The reviewer sees what changed in the new version (UC17).

UC22 · Validate and close a procedure

As a Reviewer in the final validation step, **I want** automatic checks before I close a request, **so that** I don't close one with conflicting dates or missing evidence.

Acceptance criteria

1. The final validation screen lists automatic checks, for example whether the certificate dates match the trip.
2. A failed check is shown in red and bold. The field shows the solid red label «Não coincide com a deslocação».

3. If the dates conflict, closing requires a mandatory confirmation, which explains that it is needed to close.
4. The final validator can also return or reject the request (UC18).
5. Closing generates the final document, for example the reimbursement order, and archives the procedure.
6. The notice «Procedimento fechado» appears with «Ver documento».
7. The closure is logged in the history (UC25).

UC23 · Download a generated document

As a Workflow Participant, **I want** to see and download the final document of a closed request as a PDF, **so that** Finance can use it outside defiro.

Acceptance criteria

1. Once a procedure is closed, its participants can open the final document, for example the requester from «Os meus pedidos».
2. The document opens on screen. «Fechar» and «Transferir PDF» sit in a bar fixed at the top, so they stay visible on any screen size.
3. «Transferir PDF» downloads the document as a PDF and confirms on screen that it was saved.
4. The document shows each person with their name and job title.
5. Only participants of that procedure can open its document.

UC25 · View a procedure's audit log and history

As a Workflow Participant or Workflow Manager, **I want** to see everything that happened to a request, **so that** I know who did what and when.

Acceptance criteria

1. The request page has a «Histórico do pedido · N registos» section, closed by default.
2. The «Ver histórico» button opens it and changes to «Esconder histórico».
3. Each entry shows the date and time, the person's name and job title, and the action: submitted, approved, returned with its reasons, rejected, resubmitted or closed.
4. The newest entry comes first.
5. Entries cannot be edited or deleted.
6. Participants only see the history of procedures they take part in.

UC27 · View my task inbox

As a Workflow Participant, **I want** one list of the tasks waiting for me, **so that** I know what to do next and by when.

Acceptance criteria

1. The inbox is the first screen after signing in.
2. Each task shows the request reference, the requester's name and job title, the workflow, the step and the due date.
3. A task opens in one click.
4. Returned requests appear in the requester's inbox as a task to correct (UC20).
5. When there are no tasks, the inbox says so.
6. After a decision, the user returns to the inbox, where the notice of what they did appears.
7. Deadlines and states are shown with text and an icon, not with colour alone.